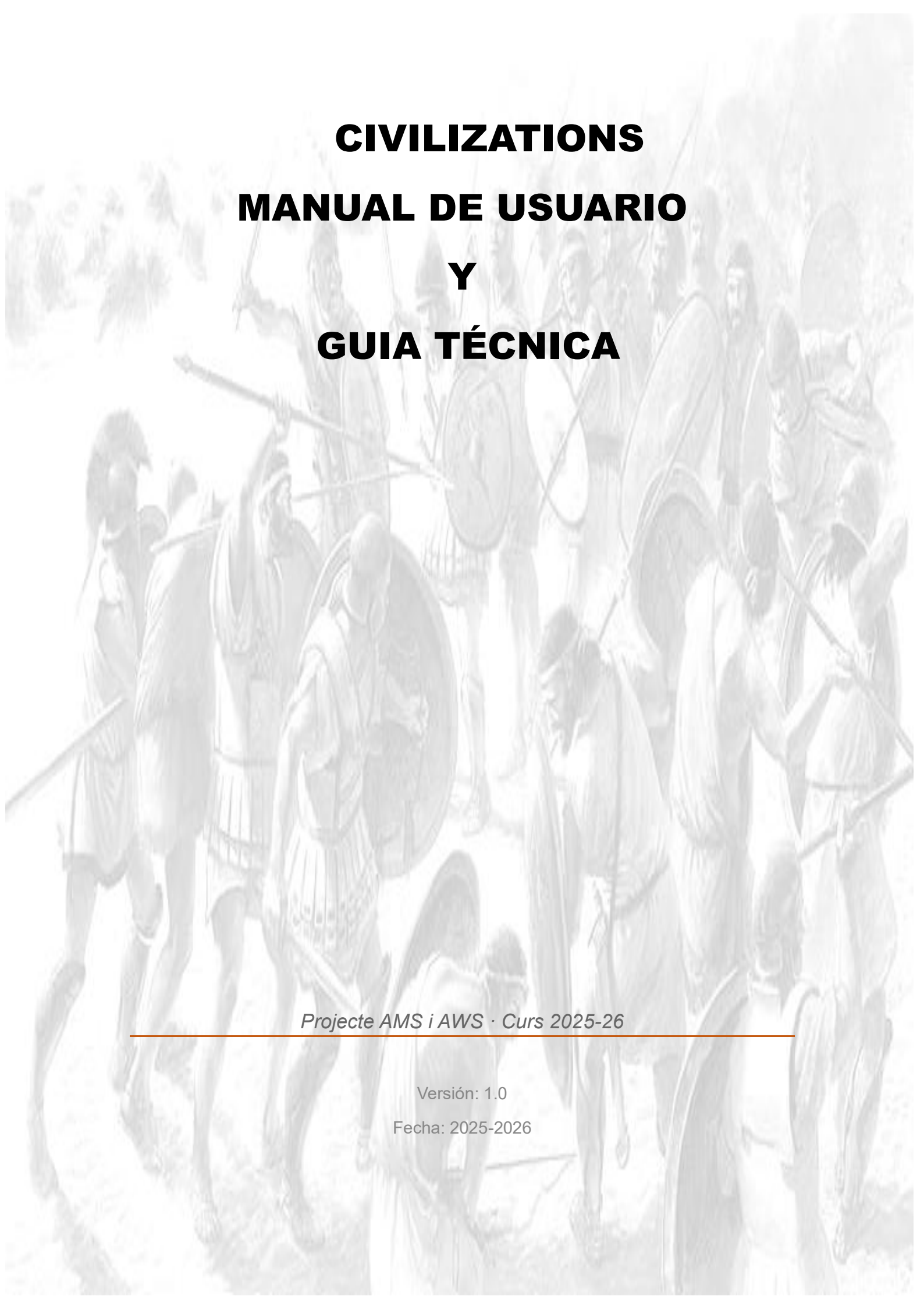




CIVILIZATIONS MANUAL DE USUARIO Y GUIA TÉCNICA

Projecte AMS i AWS · Curs 2025-26

Versión: 1.0

Fecha: 2025-2026

ÍNDICE

1. Introducción
2. Objetivo de la aplicación
3. Requisitos del sistema
4. Instalación y configuración
5. Estructura e inicio del proyecto
6. Arquitectura del sistema
7. Mecánica del juego
8. Arquitectura de clases
9. Base de datos
10. Aplicación web
11. Control de versiones (GitHub)
12. Posibles mejoras

1 INTRODUCCIÓN

Civilization es un juego de estrategia por consola (con interfaz gráfica opcional) en el que el jugador dirige una pequeña civilización. La misión principal es sobrevivir a los ataques continuos de ejércitos enemigos gestionando con eficiencia recursos, edificios, tecnologías y unidades militares.

Cada 3 minutos un nuevo ejército enemigo es generado y ataca la civilización. El jugador debe invertir en defensas y unidades ofensivas, investigar tecnologías y construir edificios para resistir el asedio. Tras cada batalla se generan escombros (madera y hierro) que pueden ser recogidos si el jugador gana.

2 OBJETIVO DE LA APLICACIÓN

El objetivo principal es gestionar una civilización y sobrevivir a invasiones enemigas mediante:

- Estrategia
- Gestión de recursos
- Creación de unidades
- Mejora tecnológica
- Administración militar

El sistema también permite visualizar información desde un dashboard web conectado a MySQL.

3 REQUISITOS DEL SISTEMA

Requisito	Mínimo	Recomendado
JDK (Java)	JDK 11+	JDK 17 o superior
SGBD	Oracle XE 21c	Oracle XE 21c / 19c
Node.js	v16+	v18 LTS
Git	2.x	última versión
RAM	4 GB	8 GB
Disco	2 GB libres	5 GB libres
SO	Windows 10 / Ubuntu 20.04	Windows 11 / Ubuntu 22.04

Dependencias necesarias:

Backend Java

- MySQL Connector/J

También se necesita acceso a un entorno Proxmox (o servidor propio) para desplegar la web con Node.js y la base de datos Oracle.

4 INSTALACIÓN Y CONFIGURACIÓN

4.1 Clonar el repositorio

Abre una terminal y ejecuta:

```
git clone https://github.com/<usuario>/civilizations.git
```

El repositorio tendrá la siguiente estructura de carpetas obligatoria:

- M01/ — Documentación y manual de usuario
- M02/ — Scripts SQL de la base de datos
- M03/ — Código fuente Java
- M04/ — Páginas web (Node.js + Handlebars)
- M05/ — Diagrama de clases y configuración de versiones
- README.md — Descripción general e instrucciones rápidas

4.2 Base de datos Oracle

La persistencia del juego se gestiona con Oracle Database XE. Sigue estos pasos:

1. Instala MySQL 21c y crea un usuario de base de datos.
2. Ejecuta los scripts de la carpeta M02/ en SQL*Plus o SQL Developer:

```
@M02/create_tables.sql @M02/insert_defaults.sql
```

3. Anota el host, puerto (por defecto 1521), SID/SERVICE_NAME, usuario y contraseña.
4. Actualiza el fichero de configuración de la aplicación Java (M03/config.properties) con esos valores.

4.3 Aplicación Java

La aplicación principal está desarrollada en Java. Para compilar y ejecutar:

```
cd M03/ javac -cp ojdbc11.jar src/**/*.java -d out/ java -cp out:ojdbc11.jar Main
```

El driver JDBC de Oracle (ojdbc11.jar) debe estar en la misma carpeta o en el classpath. Se puede descargar desde la web oficial de Oracle.

4.4 Servidor web (Node.js / HBS)

La interfaz web se sirve desde un servidor Node.js utilizando Express y plantillas HBS (Handlebars). Para instalar las dependencias y arrancar el servidor:

```
cd M04/ npm install node app.js
```

También puede ejecutarse en modo desarrollo con:

```
npm run dev
```

Por defecto el servidor escucha en el puerto 3000.

Asegúrate de que la cadena de conexión a Oracle en M04/.env o M04/config.js apunta a la misma instancia que usa la aplicación Java.

Para desplegarlo en Proxmox, configura un servicio systemd o usa PM2:

```
npm install -g pm2 pm2 start app.js --name civilizations-web pm2 save  
&& pm2 startup
```

Despliegue en Proxmox

Para mantener el servidor activo en producción se recomienda utilizar PM2.

Instalación y ejecución:

```
npm install -g pm2  
pm2 start app.js --name civilizations-web  
pm2 save  
pm2 startup
```

5 ESTRUCTURA DEL PROYECTO

A continuación se muestra la jerarquía de ficheros y carpetas principales del repositorio:

```
civilizations/ | README.md | M01/ | manual_usuario.docx | M02/ |
| create_tables.sql | insert_defaults.sql | M03/ | src/ |
| interfaces/ | Variables.java |
MilitaryUnit.java | units/ | AttackUnit.java |
| DefenseUnit.java | SpecialUnit.java |
Swordsman.java | Spearman.java | Crossbow.java |
| Cannon.java | ArrowTower.java |
Catapult.java | RocketLauncherTower.java |
Magician.java | Priest.java | exceptions/ |
| ResourceException.java | BuildingException.java |
Civilization.java | Battle.java | Main.java | out/ |
| ojdbc11.jar | M04/ | app.js | views/ | index.ejs |
| batallas.ejs | informe.ejs | civilitzacio.ejs |
| programadors.ejs | public/ | css/ | img/ |
| package.json | M05/ | diagrama_clases.png
```

Backend Java

Ejecutar:

Main.java

Frontend Web

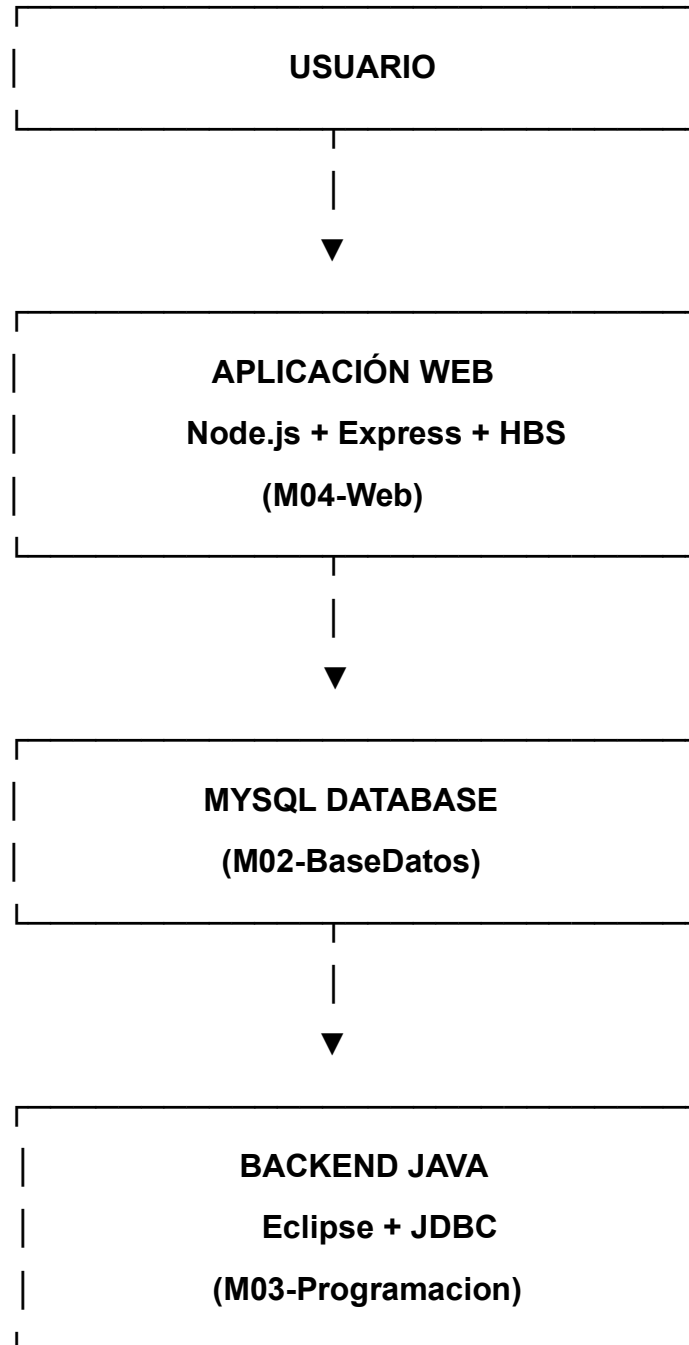
Ejecutar:

npm run dev

Abrir navegador:

http://localhost:3000

6 ARQUITECTURA DEL SISTEMA



7 MECÁNICA DEL JUEGO

6.1 Recursos

La civilización genera automáticamente recursos cada minuto mediante un TimerTask:

- Comida: 8.000 / min (base)
- Madera: 5.000 / min (base)
- Hierro: 1.500 / min (base)
- Maná: 0 (solo se genera con Torres Mágicas; 3.000 por torre y ciclo)

Los edificios incrementan la generación base. Además, al finalizar cada batalla se obtienen escombros (madera y hierro) de las unidades destruidas, si la civilización gana.

6.2 Edificios

Los edificios se construyen gastando recursos. Los costes y beneficios son:

Edificio	Comida	Madera	Hierro	Beneficio
Granja (Farm)	5000	10000	12000	+5% generación de comida
Carpintería	5000	10000	12000	+5% generación de madera
Herrería (Smithy)	5000	10000	12000	+5% generación de hierro
Torre Mágica	10000	20000	24000	Genera maná (3000/ciclo), habilita magos
Iglesia (Church)	10000	20000	24000 + 10000 maná	Permite crear 1 sacerdote por iglesia

6.3 Tecnologías

Existen dos árboles de tecnología independientes: Ataque y Defensa. Cada nivel mejora un 5% por nivel los stats correspondientes de todas las unidades nuevas creadas. El coste de subir de nivel aumenta un porcentaje fijo con cada mejora:

Tecnología	Comida	Madera	Hierro	Incremento coste
Defensa	100	200	2000	+10% comida, +15% madera, +20% hierro por nivel
Ataque	100	200	2000	+10% comida, +15% madera, +20% hierro por nivel

6.4 Unidades militares

Las unidades se dividen en tres categorías: ofensivas (army[0-3]), defensivas (army[4-6]) y especiales (army[7-8]).

Costes y estadísticas base

Unidad	Comida	Madera	Hierro	Maná	Armadura	Ataque
Swordsman	8000	3000	50	0	400	80
Spearman	5000	6500	50	0	1000	150
Crossbow	0	45000	7000	0	6000	1000
Cannon	0	30000	15000	0	8000	700
Arrow Tower	0	2000	0	0	200	80
Catapult	0	4000	500	0	1200	250
Rocket Launcher	0	50000	5000	0	7000	2000
Magician	12000	2000	0	5000	0	3000
Priest	15000	0	0	15000	0	0

Unidades ofensivas

- Swordsman: Infantería básica. Requiere comida y hierro.
- Spearman: Infantería media. Requiere comida y hierro.
- Crossbow: Arquero. Requiere madera y hierro.
- Cannon: Artillería. Requiere madera y hierro.

Unidades defensivas

- Arrow Tower: Torre de flechas. Solo requiere madera.
- Catapult: Catapulta. Requiere madera y hierro.
- Rocket Launcher Tower: Torre de cohetes. Requiere madera y hierro.

Unidades especiales

- Magician: Gran poder de ataque (3000) y alta probabilidad de repetir ataque (75%), pero 0 de armadura. Requiere al menos una Torre Mágica.

- Priest: Sin ataque ni defensa. Mientras esté vivo y haya maná, santifica las unidades ya existentes (+7% armadura y ataque). Requiere al menos una Iglesia (1 iglesia por sacerdote).

6.5 Batallas

Cada 3 minutos se genera un ejército enemigo y se inicia una batalla automáticamente. La mecánica es la siguiente:

1. Se elige aleatoriamente qué ejército ataca primero.
2. Del ejército atacante se selecciona un grupo según probabilidades predefinidas (ej. Cannon: 37% civil, 40% enemigo).
3. Del grupo se elige una unidad atacante al azar.
4. El grupo defensor se elige de forma ponderada por la cantidad de unidades de cada tipo.
5. La unidad atacante inflige daño y reduce la armadura del defensor.
6. Si la armadura llega a 0 o menos, la unidad es eliminada. Se comprueba si genera escombros (porcentaje configurado, 70% del coste en madera/hierro).
7. Se comprueba si la unidad atacante puede volver a atacar (probabilidad por tipo). De ser así, se repite desde el paso 4.
8. Si no repite, los papeles se intercambian y el proceso continúa.

La batalla termina cuando uno de los dos ejércitos tiene $\leq 20\%$ de sus unidades iniciales. El ganador es el que tenga menos pérdidas ponderadas: hierro + madera/5 + comida/10.

8 ARQUITECTURA DE CLASES

La siguiente tabla resume todas las clases e interfaces del proyecto y sus relaciones:

Clase / Interfaz	Tipo	Descripción
Variables	Interface	Constantes globales del juego (costes, stats, probabilidades)
MilitaryUnit	Interface	Contrato común para todas las unidades militares
AttackUnit	Clase abstracta	Base para unidades ofensivas; implementa MilitaryUnit y Variables
DefenseUnit	Clase abstracta	Base para unidades defensivas; implementa MilitaryUnit y Variables
SpecialUnit	Clase abstracta	Base para unidades especiales (Magician, Priest)
Swordsman	Clase concreta	Extiende AttackUnit
Spearman	Clase concreta	Extiende AttackUnit
Crossbow	Clase concreta	Extiende AttackUnit
Cannon	Clase concreta	Extiende AttackUnit
ArrowTower	Clase concreta	Extiende DefenseUnit
Catapult	Clase concreta	Extiende DefenseUnit
RocketLauncherTower	Clase concreta	Extiende DefenseUnit

Magician	Clase concreta	Extiende SpecialUnit
Priest	Clase concreta	Extiende SpecialUnit
Civilization	Clase principal	Gestiona recursos, edificios, tecnologías y ejército
Battle	Clase de lógica	Gestiona el desarrollo y reporte de cada batalla
ResourceException	Excepción	Lanzada cuando no hay recursos suficientes
BuildingException	Excepción	Lanzada cuando falta un edificio necesario
Main	Clase de entrada	Menú principal, TimerTask para recursos, batallas y amenazas

Jerarquía de herencia

El diagrama siguiente muestra cómo se relacionan interfaces, clases abstractas y clases concretas:

Clase / Interfaz	Tipo	Descripción
Variables	« <i>interface</i> »	Constantes globales del juego
MilitaryUnit	« <i>interface</i> »	Contrato de unidades militares
AttackUnit	« <i>abstract</i> »	Implementa Variables y MilitaryUnit
└ Swordsman	« <i>class</i> »	Espadachín
└ Spearman	« <i>class</i> »	Lancero
└ Crossbow	« <i>class</i> »	Ballesta
└ Cannon	« <i>class</i> »	Cañón
DefenseUnit	« <i>abstract</i> »	Implementa Variables y MilitaryUnit
└ ArrowTower	« <i>class</i> »	Torre de flechas
└ Catapult	« <i>class</i> »	Catapulta
└ RocketLauncherTower	« <i>class</i> »	Torre de cohetes
SpecialUnit	« <i>abstract</i> »	Implementa Variables y MilitaryUnit
└ Magician	« <i>class</i> »	Mago
└ Priest	« <i>class</i> »	Sacerdote
ResourceException	« <i>exception</i> »	Extiende Exception
BuildingException	« <i>exception</i> »	Extiende Exception

Métodos clave de Civilization

- newFarm(), newCarpentry(), newSmithy(), newMagicTower(), newChurch() — [construir edificios](#)
- upgradeTechnologyDefense(), upgradeTechnologyAttack() — [mejorar tecnología](#)
- newSwordsman(n), newSpearman(n), ... newPriest(n) — [añadir n unidades](#) (lanza ResourceException o BuildingException si no hay recursos/edificios)
- printStats() — muestra el estado completo de la civilización en consola

Métodos clave de Battle

- getBattleReport(int battles) — [devuelve el resumen de la batalla en String](#)
- getBattleDevelopment() — [devuelve el log paso a paso](#)
- initInitialArmies(), updateResourcesLooses(), fleetResourceCost() — [auxiliares de cálculo](#)
- getGroupDefender(), getCivilizationGroupAttacker(), getEnemyGroupAttacker() — [selección probabilística de grupos](#)
- resetArmyArmor() — [restaura armaduras tras la batalla](#)

BASE DE DATOS

La aplicación usa Oracle Database XE. Los datos se guardan y recuperan en cada sesión para poder reanudar la partida. Las tablas principales son:

Tabla	Clave primaria	Descripción
civilization_stats	civilization_id	Recursos, edificios, tecnologías y contador de batallas
attack_units_stats	civilization_id + unit_id	Tipo, armadura, daño, experiencia y santificación de unidades de ataque
defense_units_stats	civilization_id + unit_id	Ídem para unidades defensivas
special_units_stats	civilization_id + unit_id	Ídem para unidades especiales
battle_stats	civilization_id + num_battle	Madera y hierro obtenidos de cada batalla
civilization_attack_stats	civilization_id + num_battle + type	Unidades de ataque iniciales y bajas por batalla
civilization_defense_stats	civilization_id + num_battle + type	Unidades defensivas iniciales y bajas por batalla
civilization_special_stats	civilization_id + num_battle + type	Unidades especiales iniciales y bajas por batalla
enemy_attack_stats	civilization_id + num_battle + type	Unidades enemigas iniciales y bajas por batalla
battle_log	civilization_id + num_battle + num_line	Log línea a línea del desarrollo de la batalla

Los scripts de creación están en M02/create_tables.sql. Cada vez que se modifica un recurso, edificio, tecnología o se termina una batalla, la aplicación Java actualiza la base de datos mediante JDBC.

10 APLICACIÓN WEB

La web se sirve desde Node.js con Express y plantillas EJS, conectada a la misma base de datos Oracle. Debe desplegarse en el entorno Proxmox del centro. Las páginas disponibles son:

Página	Contenido
/ (Principal)	Portada, información del juego, capturas de pantalla y tabla con las 2 últimas batallas
/batalles	Listado de todas las batallas y total jugadas; enlace a informe de cada batalla
/informe?informe=ID	Informe detallado de una batalla (unidades, recursos, log)
/civilitzacio	Estadísticas de recursos en tiempo real de la civilización
/programadors	Información de los alumnos del grupo y tareas realizadas

Requisitos de la web

- Logotipo del juego visible en todas las páginas.
- Menú de navegación común hacia todas las páginas.
- Pie de página con año y nota legal/copyright.
- Diseño responsive (adaptado a móvil y escritorio).
- Animaciones CSS.
- Estética coherente con la aplicación Java.
- Entregada a través del repositorio GitHub del proyecto.

1 1 CONTROL DE VERSIONES (GitHub)

El proyecto usa Git con las siguientes convenciones obligatorias:

- Rama principal: main — solo código estable y validado.
- Rama de preproducción: preprod o develop — donde se fusionan los cambios individuales.
- Rama personal por alumno: cada miembro trabaja en su propia rama y hace pull request a preprod.
- Cada commit debe incluir un mensaje descriptivo del cambio realizado.
- Carpetas M1–M5 presentes desde el inicio del proyecto.
- README.md con descripción breve, instrucciones de instalación y ejecución.
- La entrega final se realiza como release en GitHub; la URL se sube a la tarea de Moodle de M5.

1 2 POSIBLES MEJORAS

Las siguientes funcionalidades pueden incorporarse como ampliación del proyecto:

- Coste de restaurar armaduras después de cada batalla (actualmente es gratuito).
- Ejército enemigo procedente de una civilización rival con economía propia.
- Intervalos de ataque aleatorios en lugar de fijos cada 3 minutos.
- Nivel máximo de tecnología configurable en la interfaz Variables.
- Coste de mantenimiento en comida para Swordsman y Spearman; si no hay comida, mueren unidades.
- Límite de unidades que puede santificar cada sacerdote.
- Interfaz gráfica completa (JavaFX o Swing) para controlar todas las opciones del juego.
- Sistema de guardado automático periódico además del guardado al pausar/cerrar.
- Múltiples civilizaciones jugando en el mismo servidor con ataques cruzados.